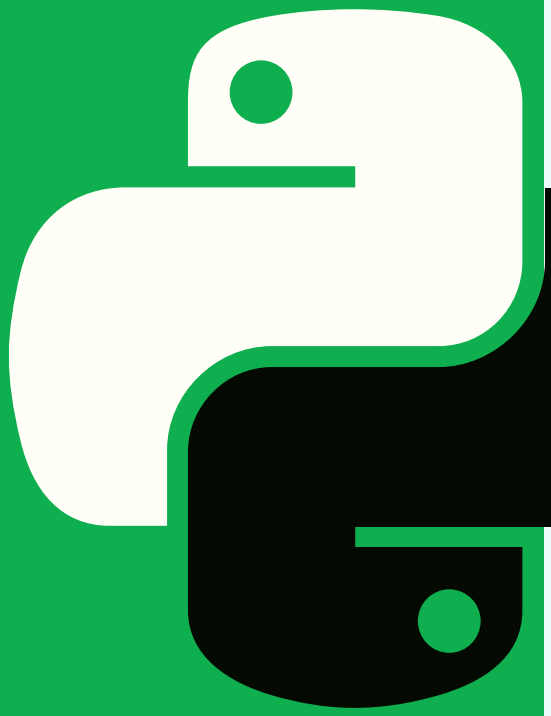
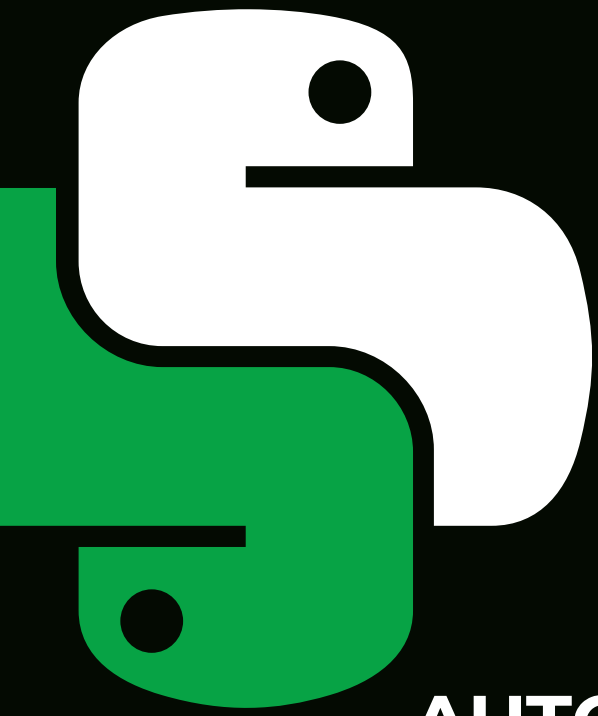
python



PARA AUTOMAÇÃO DE TAREFAS

GABRIEL PIETSIAKI

CODIFICA



PYTHON PARA AUTOMAÇÃO DE TAREFAS

CODIFICA
CNPJ 02.274.882/0001-09
Campus Universitário
86057-250
Rodovia Celso Garcia Cid, S/N
Londrina, Paraná

Sumário

0 Introdução **2**

1 Elementos básicos **2**
1.1 Terminal 2
1.2 Tipos de dados 3
1.3 Fluxo de execução 6

2 Elementos avançados **9**
2.1 Funções 9
2.2 Pacotes 9

3 Arquivos **10**
3.1 Arquivos de texto 10
3.2 Arquivos CSV 10

4 Processamento de dados **12**
4.1 Identificação de itens em diretório 12
4.2 Manipulação de pastas 12
4.3 Manipulação de múltiplos arquivos 13

5 Programas **14**
5.1 Menus de opções 14
5.2 Criação de um programa com Terminal 16

0 Introdução

A linguagem de programação Python tem se tornado um instrumento de notável poder para automatizar tarefas e solucionar problemas diários, inclusive para usuários sem experiência prévia de programação. O presente material buscaq por meio de uma abordagem didática apresentar objetivamente os fundamentos da linguagem e suas aplicações práticas, capacitando o leitor a otimizar a gestão de seu tempo e elevar sua produtividade.

Ao longo do curso, serão abordadas temáticas como escrita de código básica de código, variáveis, operadores, estruturas de controle, funções, bibliotecas e automação de tarefas. O objetivo final consiste em proporcionar ao discente autonomia para utilizar o Python como ferramenta de resolução de problemas e otimização de tarefas, transformando a maneira como ele trabalha e vive.

1 Elementos básicos

Para podermos utilizar o Python, é necessário compreender três conceitos básicos da linguagem: *valor*, *variável* e *ação*.

valor é toda informação que se opera dentro do código programado. Ao valor se atribui tipo, operações básicas, composições entre valores, entre outros.

variável é o objeto qual recebe um valor e permite o acesso a manipulação destes. São atreladas ao tipo do valor que carregam, entretanto, em Python, variáveis podem ser convertidas entre tipos.

ação é toda manipulação que atua sobre variáveis, realizando acessos aos seu valores, executando modificações, apresentando resultados, entre outros.

1.1 Terminal

Para toda linguagem de programação, o terminal é um elemento essencial de desenvolvimento de aplicações. Utilizando funções simples de entrada e de saída de dados no programa, podemos utilizar o terminal como interface na execução. Em aplicações complexas podem ser criados menus de opções para aprimorar a usabilidade do programa.

O acesso ao terminal depende da IDE (ambiente de desenvolvimento) de escolha, contudo, em geral, ao selecionar a execução do código, espera-se que uma janela do terminal seja apresentada ao usuário.

Saída de dados

Em geral, uma única função de saída de dados é suficiente para transmitir os dados ao usuário. Tal função nativa da biblioteca padrão do Python é `print()`. A variável que for dada para a função terá seu valor apresentado na tela do terminal.

EXEMPLO

```
valor = 50      # defindo um numero para variavel valor
print(valor)    # imprimindo no terminal o valor
```

```
TERMINAL

50
```

Formatações de impressões complexas podem ser obtidas utilizando a formatação de *strings*, como visto no exemplo a seguir:

EXEMPLO

```
valor = 50      # defindo um numero para variavel valor
print(f"o meu valor: {valor}")     # impressao do valor
```

```
TERMINAL

o meu valor: 50
```

Entrada de dados

O Python apresenta também uma função para obter valores de entrada pelo terminal: `input()`. Quando executada, o programa aguardará até que uma cadeia de caracteres seja digitada e, então, a tecla `Enter` pressionada.

EXEMPLO

```
usuario = input("Digite seu nome: ")
print("Bem-vindo, " + usuario)
```

```
TERMINAL

Digite seu nome: CODIFICA
Bem-vindo, CODIFICA
```

1.2 Tipos de dados

Tipos de dados numéricos

int armazena valores inteiros. Ex.: $\cdots - 3, -2, -1, 0, 1, 2, 3 \ldots$

float armazena valores com parte fracionária. Ex.: $2.5, -7.01, 0.123, 8.0$

complex armazena valores numéricos complexos (conjunto numérico matemático)

Valores numéricos permitem algumas operações básicas, estas são:

Operação	Operador	Utilização	Exemplo
Adição	`+`	`x = a + b`	`5 + 3` = 8
Multiplicação	`*`	`x = a * b`	`2 * 6` = 12
Divisão	`/`	`x = a / b`	`18 / 2` = 9
Potenciação	`**`	`x = a ** b`	`2 ** 3` = 2^3 = 8
Resto	`%`	`x = a % b`	`6 % 3` = 0

Tipos de dados não-numéricos

str armazena cadeias de caracteres, isto é, palavras ou frases. Útil para processar nomes, endereços e diversas informações textuais

bool armazena apenas binários, somente `True` (verdadeiro) ou `False` (falso). Útil para o processamento do *Fluxo de execução*

A declaração de uma variável não necessita especificar seu tipo, apenas atribuir um valor.

EXEMPLO

```
meuValorInteiro  = 5          # numero inteiro (int)
meuValorFloat    = 3.141      # numero tipo (float)
meuValorComplexo = 3+5j       # numero complexo (complex)

nomeEmpresa      = "Codifica" # string (str)
emFuncionamento  = True       # valor booleano (bool)
```

Conjuntos de dados

Permite armazenar múltiplos dados sem a necessidade de criar variáveis individuais para cada valor, mas somente um nome que representa aquele conjunto. Por exemplo, uma lista de compras, onde itens podem ser adicionados, removidos, reordenados ou processados.

Python permite que sejam criados conjuntos de dados simples utilizando listas (*lists*) ou tuplas (*tuples*), suas diferenças são bem simples, pois realizam tarefas semelhantes.

Uma *List* armazena valores de tipos variados sequencialmente, podemos acessar individualmente cada elemento dentro da lista por meio do seu **índice** e também realizar operações de manipulação da lista (inserção, remoção, ordenação, etc) por meio de funções especiais.

O funcionamento de **listas** é dado por:

CRIAÇÃO para definir uma lista, usa-se sempre colchetes `[ ]` com os elementos dentro separados por vírgula ou a função `list()`. Exemplo: `linguagens = ["Python", "C"]`

ÍNDICES toda *List* é ordenada por meio de índices, cada elemento possui um índice associado referente a sua posição da lista. O índice de uma lista começa em 0, então uma lista de 3 elementos tem índices: $0, 1$ e 2.

ACESSO para obter ou alterar o valor de um item de uma *List*, utiliza-se o nome da lista seguido do índice desejado entre colchetes. Exemplo: `compras[0]`

EXEMPLO

```
notas = [8, 5, 2, 1]      # criacao de uma List de inteiros
print(notas[0], notas[1]) # primeira escrita no terminal
notas[0] = 10             # manipulacao de valor
notas[1] = notas[2]       # manipulacao de valor
print(notas[0], notas[1]) # segunda escrita no terminal
```

```
TERMINAL

8 5
10 2
```

Por sua vez, **tuplas** realizam exatamente o mesmo trabalho que listas permitem, porém não é permitido realizar modificação dos valores dentro de uma tupla. Uma vez criada, esta é imutável. Diferentemente de listas, tuplas são criadas com parenteses `( )`, contudo seguem o mesmo sistema de índices e de acesso de listas.

Algumas funcionalidades básicas para o processamento de listas podem ser utilizadas por métodos nativos da linguagem, são:

Método	Descrição
`append`	adiciona um elemento ao fim da lista
`pop`	retira o elemento presente no dado índice e retorna
`index`	retorna o índice da primeira ocorrência de um valor
`sort`	ordena a lista em ordem crescente ou decrescente
`copy`	produz uma cópia idêntica da lista e retorna

A utilização dos métodos é dada ao inserir um ponto `'.'` após o nome da variável do tipo *List*, seguido do método.

EXEMPLO

```
valores = [10,4,1,7] # criacao de uma List de inteiros
valores.append(15)   # insercao do valor 15
print("tamanho da lista: ", len(valores))
valores.sort()       # metodo para ordenacao ascendente
print("lista: ", valores)
```

```
TERMINAL

tamanho da lista:  5
lista:  [1, 4, 7, 10, 15]
```

Veja códigos completos sobre este assunto!

REPOSITÓRIO ACESSAR EXEMPLO

1.3 Fluxo de execução

Testes condicionais

Serve para comparar dados, o resultado de um teste condicional é sempre um tipo booleano, por exemplo: verificar se um número é maior ou menor que outro; comparar duas strings; testar igualdade entre números; entre outros. Cada uma destas comparações produz somente dois resultados: ou a afirmação é verdadeira, ou é falsa.

Palavra-chave: `if`

O código dentro de um bloco *if* só é executado quando um teste **`<condicao>`** é verdadeiro durante a execução. *If* é o elemento mais básico do fluxo de execução de um código, pois permite fragmentar a execução em partes específicas do código conforme condições programadas.

SINTAXE

```
if (condicao):
        # bloco de codigo
```

EXEMPLO

```
valor = 78
if (valor > 0):
        print("positivo")
if (valor < 0):
        print("negativo")
if (valor == 0):
        print("neutro")
```

```
TERMINAL

positivo
```

Palavra-chave: `else`

a palavra-chave *else* é um complemento ao uso de *if*, serve para reservar a execução ao caso contrário do teste, ou seja, quando o teste **`<condicao>`** é dado como **`False`**. Útil quando há um tratamento específico tanto para o sim, quanto para o não.

SINTAXE

```
if (condicao):
        # bloco de codigo para caso seja verdadeiro
else:
        # bloco de codigo para caso seja falso
```

EXEMPLO

```
valor = 78
if (valor > 0):
        print("positivo")
else:
        if (valor < 0):
                print("negativo")
        else:
                print("neutro")
```

```
TERMINAL

positivo
```

Laços de repetição

São usados para repetir uma sequência de instruções por um número definido de vezes.

Palavra-chave: `for`

Um laço de repetição *for* percorre uma lista, tupla e outros tipos de coleções passando item a item. Isto ocorre por meio de uma variável **`<item>`** utilizada para receber cada uma das unidades presentes em **`<colecao>`**.

SINTAXE

```
for item in colecao:
        # bloco de codigo
```

EXEMPLO

```
for numero in [1, 2, 3]:
        print(numero)
```

```
TERMINAL

1
2
3
```

EXEMPLO

```
listaDeCompras = ["abacaxi", "banana", "cenoura"]
for comida in listaDeCompras:
        print(f"- {comida}")
```

```
TERMINAL

- abacaxi
- banana
- cenoura
```

Palavra-chave: `while`

Do inglês "enquanto", este laço realizará repetições de um bloco de instruções enquanto uma dada afirmação **`<condicao>`** for testada e verdadeira. Diferentemente do *for*, o *while* é um laço de repetição útil para quando a quantidade de repetições dependa de um fator lógico.

SINTAXE

```
while (condicao):
    # bloco de codigo
```

EXEMPLO

```
contador = 0
while (contador < 3):
    print(contador)
    contador += 1
```

```
TERMINAL

0
1
2
```

Veja códigos completos sobre este assunto!

REPOSITÓRIO ACESSAR EXEMPLO

2 Elementos avançados

2.1 Funções

São úteis para tornar o código menos repetitivo, mais abstrato e condensado. Por exemplo, uma função que é utilizada com muita frequência é **`print()`**, uma função nativa da biblioteca básica de Python para escrever textos no terminal. Utilizando uma biblioteca matemática.

Uma função se assemelha com uma máquina, onde esta recebe um ou mais parâmetros quaisquer e pode, ou não, retornar algum resultado. Quando uma função deve retornar algo, utiliza a palavra-chave **`return`** seguida dos valores de retorno. Ao atingir a instrução **`return`**, a função é imediatamente finalizada e retorna ao ponto de origem, onde foi chamada.

Definir uma função é escrever quais são as instruções que a função deve executar quando algo a chamar.

SINTAXE

```
def nome(parametro1, parametro2, parametro3, ... ):
        # bloco de codigo
        return resultado
```

EXEMPLO

```
def jurosSimples(capital, taxa, tempo):
    return capital * taxa * tempo
def montante(capital, taxa, tempo):
    return capital + jurosSimples(capital, taxa, tempo)
```

2.2 Pacotes

Pacotes permitem complementar o nível de programação do Python por meio da importação de módulos providos por terceiros. Funções que não existem nativamente na linguagem podem ser encontradas em pacotes, como diversos que são apresentados nesta apostila.

Instalação de pacotes

Cada pacote importado possui sua forma de instalação desejada, ou mesmo seleção de tipo de instalação, entretanto, é comum que o seja realizado por meio do gerenciador de pacotes **`pip`**.

A instalação de um pacote na máquina só é necessária uma única vez e ocorre, em geral, pelo comando no terminal da seguinte forma:

```
TERMINAL

pip nome_do_pacote
```

Veja códigos completos sobre este assunto!

REPOSITÓRIO ACESSAR EXEMPLO

3 Arquivos

3.1 Arquivos de texto

Com arquivos de textos, uma aplicação pode ler, processar e guardar informações que também podem ser alteradas por uma pessoa. Arquivos de texto simples como `.txt` podem ser abertos com blocos de notas e são facilmente manipuláveis por linhas de código.

Leitura

Utilizando a função `open()` em modo de leitura podemos abrir o arquivo de texto localizado em `<caminho>`. Em seguida, os métodos `readline()`, para ler uma linha, e `readlines()`, para ler todas as linhas, podem ser utilizados para obter o conteúdo dos arquivos de texto.

CASO GERAL

```
with open(caminho, mode = 'r') as arquivoTexto:
    linhasLidas = arquivoTexto.readlines()
    for linha in linhasLidas:
        # bloco de codigo
```

Veja códigos completos sobre este assunto!

REPOSITÓRIO ACESSAR EXEMPLO

Escrita

Utilizando a função `open()` em modo de escrita podemos abrir o arquivo de texto localizado em `<caminho>`. Em seguida, os métodos `write()`, para escrever uma linha, e `writelines()`, para escrever múltiplas linhas, podem ser utilizados para escrever dados `<conteudo>` nos arquivos de texto.

CASO GERAL

```
with open(caminho, mode = 'w') as arquivoTexto:
    write(conteudo)
```

Veja códigos completos sobre este assunto!

REPOSITÓRIO ACESSAR EXEMPLO

3.2 Arquivos CSV

Ferramenta de tabelas e de grande processamento de dados geralmente permitem a exportação de seus arquivos em formato CSV, *Comma Separated Values*. Arquivos CSV são nada mais que arquivos de texto padronizados para armazenar valores em linhas e colunas. Pode-se

utilizar deste tipo de arquivo para inserir dados reais no programa, realizando processamento adicional, ou mesmo exportando dados obtidos durante a execução para um arquivo CSV e, então, manipular com maior facilidade em outra ferramenta.

Python possui um módulo nativo para este tipo de manipulação de arquivos, o módulo **`csv`**, para utilizar, deve-se realizar a importação no cabeçalho do arquivo por meio do comando

```
import csv
```

Leitura

A leitura de um arquivo CSV é simples, dado o diretório **`<caminho>`** onde o arquivo CSV se encontra, realiza-se a abertura do arquivo por meio da função **`open()`** utilizando o modo de leitura. Por fim, pode-se ler o arquivo linha a linha realizando iterações sobre todas as linhas lidas em **`<arquivoCSV>`**. Cada **`<linha>`** é uma *List* de strings dos valores contidos das células daquela linha, o número de índices que cada linha possui é dado pela quantidade de colunas da tabela.

CASO GERAL

```
import csv
with open(caminho, mode = 'r') as arquivo:
    arquivoCSV = csv.reader(arquivo)
    for linha in arquivoCSV:
        # bloco de codigo
```

Veja códigos completos sobre este assunto!

REPOSITÓRIO ACESSAR EXEMPLO

Escrita

Semelhante a leitura, a escrita de um arquivo CSV também ocorre linha a linha. Para a escrita, o programa criará um arquivo com dado nome em **`<caminho>`** e abrirá para escrita. Então, utilizando os métodos **`writerow()`** e **`writerows()`** o arquivo é construído. O método **`writerow()`** escreverá uma linha no arquivo CSV ao fornecer uma lista (*List*) com valores. Por sua vez, o método **`writerows()`** agiliza o trabalho, pois, escreve todas as linhas fornecidas dentro de uma lista, o que então seria uma lista de listas.

CASO GERAL

```
import csv
with open(caminho, mode = 'w') as arquivo:
    arquivoCSV = csv.writer(arquivo)
    arquivoCSV.writerow(cabecalho)
    arquivoCSV.writerows(tabela)
```

Veja códigos completos sobre este assunto!

REPOSITÓRIO ACESSAR EXEMPLO

4 Processamento de dados

4.1 Identificação de itens em diretório

Utilizando os dados de todos os itens presentes dentro de um diretório fornecidos pela função **os.listdir()**, obtem-se uma lista de strings contendo os nomes de cada um dos itens que estão presentes no dado diretório fornecido, isto é, pastas (diretórios) e arquivos. Ao omitir o diretório desejado, a função **os.listdir()** considera o diretório corrente de execução, este, por padrão, é dado pelo local onde o código Python foi executado.
CASO GERAL: obtendo itens do diretório corrente

```
import os
itens = os.listdir()
```

As funções **os.path.isfile()** e **os.path.isdir()** podem ser aplicadas sobre um item para identificar o seu tipo: arquivo ou diretório. Desta forma, filtragens podem ser feitas na lista de itens obtida de maneira a selecionar somente arquivos ou pastas contidas no diretório.
CASO GERAL: filtrando itens do diretório corrente

```
import os
itens = os.listdir()

# Lista somente dos arquivos
arquivos = [item for item in itens if os.path.isfile(item)]
pastas   = [item for item in itens if os.path.isdir(item)]
```

4.2 Manipulação de pastas

Criação

Um diretório pode ser criado dentro do diretório corrente de execução por meio do comando **os.mkdir()**.
CASO GERAL

```
import os
os.mkdir("nome_da_pasta")
```

Exclusão

As funções **os.rmdir()** e **os.remove()** atuam respectivamente na exclusão de diretórios e de arquivos. Estas realizam suas operações sobre o dado nome de pasta ou arquivo especificado como parâmetro. Em especial, a função de remoção de diretório necessita que a pasta alvo esteja completamente vazia para que então atue.

CASO GERAL

```
import os
os.remove("pasta/arquivo") # remove o arquivo
os.rmdir("pasta")          # remove o diretorio
```

4.3 Manipulação de múltiplos arquivos

O processo para movimentar arquivos entre diretórios do computador pode ser realizado utilizando a função **`shutil.move()`** do módulo *shutil*. A função realiza a movimentação de um dado arquivo localizado em um diretório **`<origem>`** para um diretório **`<destino>`**.
CASO GERAL: movendo todos os arquivos de uma pasta para outra

```
import shutil
import os

files = os.listdir(origem)
for arquivo in listagemArquivos:
    nomeCompleto = os.path.join(origem, arquivo)
    shutil.move(nomeCompleto, destino)
```

5 Programas

5.1 Menus de opções

A utilização de menus em programas de terminal torna prática a usabilidade, para isto, pode-se usar o pacote *inquirer* ou *simple_term_menu* para criação de menus com interação. Ambos os pacotes apresentam várias opções de customização, porém antes de construir um menu é necessário conhecer:

1. Quais são as opções?
2. Pode-se escolher mais de uma opção?
3. Como o menu deve se comportar após a escolha?

Pacote: *inquirer* (Multiplataforma)

Tendo conhecimento do funcionamento desejado do menu, para todos os casos é preciso definir quais são as opções a apresentar no menu, estas devem colocadas como *strings* em uma *List* **`<opcoes>`**. Posteriormente, cria-se uma variável que representa a pergunta a ser realizada **`<pergunta>`** por meio de **`inquirer.List`**. Por fim, a resposta obtida é retornada após a execução de:

```
inquirer.prompt(pergunta)
```

Instalação

```
pip install inquirer
```

Importação

```
import inquirer
```

Utilização Pode-se utilizar o menu por meio das setas do teclado para alternar entre as opções e **`Enter`** para selecionar a opção.
CASO GERAL: menu de seleção única

```
import inquirer
opcoes   = ["opcao A", "opcao B", "opcao C"]
pergunta = [
    inquirer.List("valor",
                  message = "O que deseja fazer?",
                  choices = opcoes
    )
]
resposta = inquirer.prompt(pergunta)
# bloco de codigo
```

Seleção múltipla Para o caso de menu com seleções múltiplas de opções, utiliza-se o tipo `inquirer.Checkbox` ao invés de `inquirer.List`. Agora, as opções são selecionáveis pela tecla de espaço e `Enter` para finalizar as seleções. O retorno obtido é uma lista com os índices selecionados.
CASO GERAL: menu de seleção múltipla

```
import inquirer
opcoes   = ["opcao A", "opcao B", "opcao C"]
pergunta = [
    inquirer.Checkbox("valor",
                message = "O que deseja fazer?",
                choices = opcoes
    )
]
respostas = inquirer.prompt(pergunta)
# bloco de codigo
```

Pacote: *simple_term_menu* (Linux)

Tendo conhecimento do funcionamento desejado do menu, para todos os casos é preciso definir quais são as opções a apresentar no menu, estas devem colocadas como *strings* em uma *List* `<opcoes>`. Posteriormente, cria-se uma variável que representa o menu `<menu>` por meio de `TerminalMenu(opcoes)`. Por fim, a(s) opção(ões) escolhida(s) são retornadas como índice referentes à(s) opção(ões) da lista de opções após utilizar a função `menu.show()`.

Instalação

```
pip install simple-term-menu
```

Importação

```
from simple_term_menu import TerminalMenu
```

Utilização Pode-se utilizar o menu por meio das teclas `j` e `k` do teclado para alternar entre as opções e `Enter` para selecionar a opção. É possível também selecionar opções por meio de atalhos atribuindo a tecla desejada entre colchetes `[ ]` no prefixo da opção.
CASO GERAL: menu de seleção única com atalhos

```
from simple_term_menu import TerminalMenu

opcoes = ["[0] opcao A", "[1] opcao B", "[2] opcao C"]
menu   = TerminalMenu(opcoes, title = "titulo")
indice = menu.show()
# bloco de codigo
```

Seleção múltipla Para o caso de menu com seleções múltiplas de opções, defini-se o parâmetro `multi_select` para `True` na construção do menu. Agora, as opções são selecionáveis pela tecla de espaço e `Enter` para finalizar as seleções. O retorno obtido é uma lista com os índices selecionados.
CASO GERAL: menu de seleção múltipla

```
from simple_term_menu import TerminalMenu

opcoes  = ["opcao A", "opcao B", "opcao C"]
menu    = TerminalMenu(opcoes, title = "titulo",
                       multi_select = True,
                       show_multi_select_hint = True)
indices = menu.show()
# bloco de codigo
```

5.2 Criação de um programa com Terminal

Procedimentos utilitários

Pressionar tecla para continuar Um simples procedimento para aguardar o usuário confirmar a leitura dos dados é requisitar que pressione alguma das teclas do teclado para que a execução do programa continue. Para isto, pode-se utilizar o pacote *keyboard*, este deve ser instalado por meio do comando:

```
pip install keyboard
```

CASO GERAL

```
import keyboard

# bloco de codigo anterior

print("Pressione alguma tecla para continuar...")
keyboard.read_key(suppress=True)

# bloco de codigo posterior
```

Executar programas como aplicativos

Tendo um código funcional que apresenta opções de entrada e saída de dados pelo terminal, pode-se facilmente transformar este código em um aplicativo, mantendo suas funcionalidades de terminal. Para isto, basta realizar dois cliques sobre o arquivo de extensão `.py` que contém o código, porém é necessário pedir para que o programa espere um momento (*Pressionar tecla para continuar*) antes da sua finalização, pois uma vez que aconteça, o terminal é finalizado **automaticamente**.

Organização Manter todos os códigos de programas em uma pasta facilmente localizável é útil para manter o controle dos códigos existentes e realizar ajustes com facilidade.

Atalho e Ícone Para evitar comprometer os códigos desenvolvidos, ao invés de executar os programas diretamente pelos arquivos Python, pode-se criar atalhos no Windows com o botão direito e selecionando a opção "Criar atalho", este pode ser movido e colocado em qualquer lugar do computador sem comprometer seu código original. Ainda, nas propriedades do atalho, é possível alterar seu ícone escolhendo dentre as opções já existentes ou obtendo ícones da internet em formato `.ico`.